

SYSTEM DESIGN — HLD

Video 3: API Gateway vs Load Balancer

Video 4: Load Balancer, Types & Algorithms

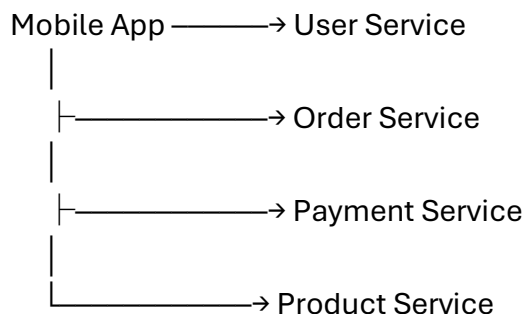
PART 1 — API GATEWAY

1. What is an API Gateway?

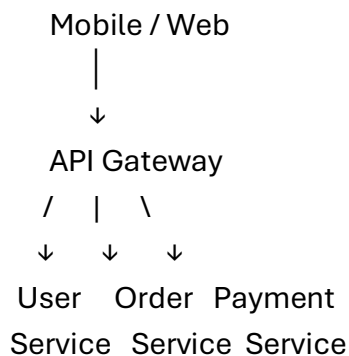
An **API Gateway** is a server/component that acts as an **entry point between clients and backend services**.

In a microservices architecture, instead of the client directly communicating with every service:

✗ Without API Gateway



We can introduce an API Gateway:



So:

API Gateway = a single entry point through which clients access multiple backend services.

2. Why Do We Need an API Gateway?

Imagine an application with:

User Service
Order Service
Payment Service
Product Service
Notification Service

Without a gateway, the client needs to know:

- Where each service is located
- Which API to call
- How authentication works for each service
- How to handle service failures
- How to communicate with different services

This creates complexity on the client side.

With API Gateway:

Client
↓
API Gateway
↓
Correct Service

The client only needs to know about the gateway.

3. Main Responsibilities of API Gateway

Your notes mention three important features. There are several others worth knowing.

3.1 Request Routing ★

The gateway determines where a request should go.

Example:

GET /users/123



API Gateway



User Service

And:

POST /orders



API Gateway



Order Service

3.2 Authentication & Authorization

The gateway can verify whether the client is allowed to access the system/service.

Example:

Client



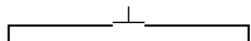
API Gateway



Validate Token



Valid?



Yes

No



Service

401/403

Authentication

Who are you?

Example: Verify JWT.

Authorization

What are you allowed to access?

Example:

Admin → /admin/users ✓

User → /admin/users ✗

3.3 Request Aggregation ★

This is one of the useful features you noted as multiple APIs.

Suppose a mobile app needs:

User information

+

Order information

+

Recommendation information

Without aggregation:

Mobile App

└─→ User API

└─→ Order API

└─→ Recommendation API

With API Gateway:

Mobile App

↓

API Gateway

/ | \

↓ ↓ ↓

User Order Recommendation

\ | /

\ | /

└─┬─┬─┘

↓

Combined Response

↓

Client

This reduces the number of requests the client needs to make.

Example

Instead of:

GET /user
GET /orders
GET /recommendations

The client could call:

GET /home

and the gateway internally calls multiple services and combines the results.

3.4 Rate Limiting

The gateway can restrict how many requests a client can make.

Example:

User

↓

API Gateway

↓

100 requests/minute allowed

If the user exceeds the limit:

429 Too Many Requests

You'll study Rate Limiting much later in your playlist, so remember only the basic connection for now.

3.5 SSL/TLS Termination

HTTPS encryption can be handled at the gateway.

Client

|

| HTTPS

↓

API Gateway

|

| HTTP/internal HTTPS

↓

Backend Services

This is called **TLS/SSL termination**.

You'll learn SSL/TLS separately in your playlist.

3.6 Logging & Monitoring

The gateway can provide a centralized place for:

- Request logs
- Response status
- Latency
- Errors
- Traffic metrics

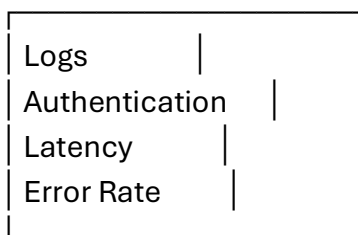
Example:

Client

↓

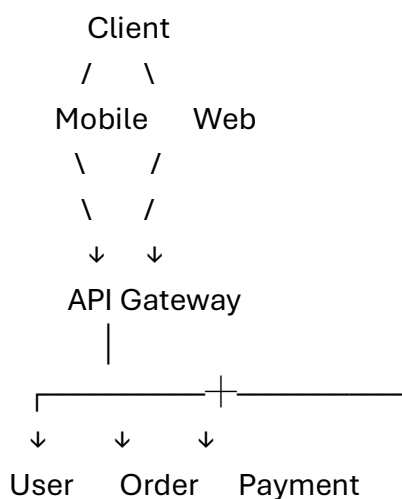
API Gateway

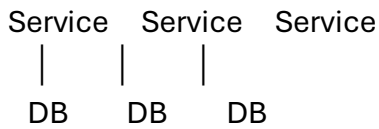
↓



4. API Gateway — Complete Flow

Keep this flow in your notes:





The API Gateway can perform:

Request



Authentication



Authorization



Rate Limiting



Routing



Service



Response

PART 2 — LOAD BALANCER

5. What is a Load Balancer?

A **Load Balancer (LB)** distributes incoming network traffic across multiple servers/instances.

Suppose we have:

1000 requests

and three servers:

Server 1

Server 2

Server 3

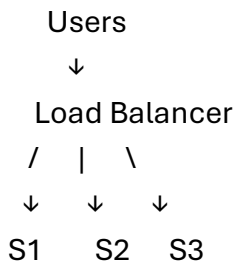
Instead of sending everything to one server:

1000 requests



Server 1

we distribute the traffic:

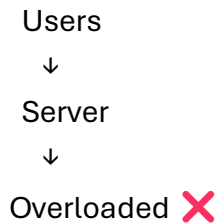


The goal is to prevent one server from becoming overloaded.

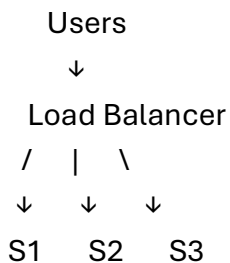
6. Why Do We Need Load Balancing?

Problem 1 — Overloading

Without LB:



With LB:



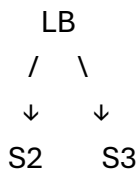
Traffic is distributed.

Problem 2 — High Availability

Suppose:

- S1 ❌
- S2 ✅
- S3 ✅

The load balancer can stop sending traffic to S1.



Users can continue using the application.

Problem 3 — Horizontal Scaling

When traffic increases:

100 users



2 servers

Later:

1 Million users



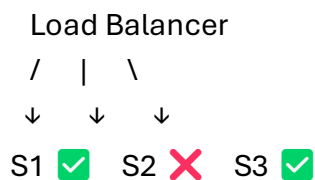
10 servers

The load balancer distributes requests among the new instances.

7. Health Checks ★

A load balancer should know whether a backend server is healthy.

Example:



The LB detects that S2 is unhealthy and stops routing traffic to it.

Usually this is done using **health-check endpoints**.

Example:

GET /health

Response:

200 OK

means the instance is healthy.

PART 3 — API GATEWAY vs LOAD BALANCER

This is one of the **most important concepts from Video #3**.

They can both sit in front of backend systems, but they solve different problems.

8. API Gateway vs Load Balancer

API Gateway

Primarily deals with:

Client → Services

It understands things like:

- API routes
- Authentication
- Authorization
- Rate limiting
- Request transformation
- Request aggregation

Load Balancer

Primarily deals with:

Traffic → Instances

It decides which server/instance should receive the request.

Example

Suppose:

User
↓
API Gateway
↓
Order Service

But Order Service has 3 instances:

Order Service
├— Instance 1
├— Instance 2
└— Instance 3

Now:

User
↓
API Gateway
↓
Order Service
↓
Load Balancer
/ | \
↓ ↓ ↓
O1 O2 O3

The gateway decides:

"This request belongs to Order Service."

The load balancer decides:

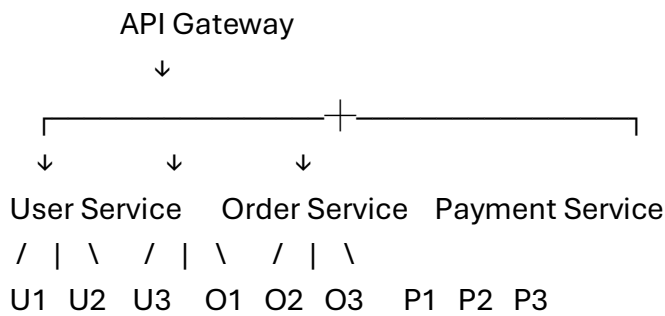
"Which Order Service instance should handle it?"

9. Can API Gateway and Load Balancer Work Together?

Yes. Very commonly.

A typical architecture can look like:

Clients
↓
Load Balancer
↓



Here:

Load Balancer

Distributes traffic among API Gateway instances.

API Gateway

Routes requests to the correct service.

Service-level Load Balancer

Can distribute traffic among instances of a particular service.

The exact architecture depends on the infrastructure.

10. Important Correction

Don't memorize:

"API Gateway is the entry point and Load Balancer isn't."

That's not always true.

Both can act as traffic-entry components depending on architecture.

A better statement is:

API Gateway provides API-level functionality and routing, while a Load Balancer distributes traffic among available instances.

PART 4 — TYPES OF LOAD BALANCERS

There are two major types you should know.

11. Layer 4 Load Balancer — L4

L4 operates at the **Transport Layer**.

It mainly works with:

- TCP
- UDP
- IP
- Port

It doesn't need to understand the actual HTTP request content.

Example:

Client

↓

L4 Load Balancer

↓

Server

It may route based on information such as:

Source IP

Destination IP

Port

Protocol

Advantages

- Fast
- Low overhead
- Good for high-throughput TCP/UDP traffic

Use cases

Useful when you need to balance:

- TCP connections
- UDP traffic
- Non-HTTP applications

12. Layer 7 Load Balancer — L7

L7 operates at the **Application Layer**.

It can understand protocols such as HTTP/HTTPS.

Therefore, it can make routing decisions based on:

- URL path
- HTTP method
- Headers
- Cookies
- Hostname

Example:

Request:

GET /payments

↓

L7 Load Balancer

↓

Payment Service

Another:

GET /users

↓

L7 Load Balancer

↓

User Service

13. L4 vs L7

L4 Load Balancer

Transport layer

TCP/UDP/IP

Less request information

Faster/lower overhead

Routing based on IP/port
etc.

Useful for TCP/UDP

L7 Load Balancer

Application layer

HTTP/HTTPS

Understands HTTP requests

More processing

Routing based on URL, headers, host,
etc.

Useful for web/API traffic

Easy memory trick

L4 → Connection-level

L7 → Application/request-level

PART 5 — LOAD BALANCING ALGORITHMS

Now we reach the main part of **Video #4**.

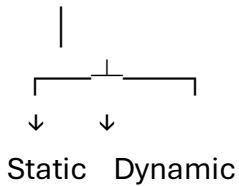
A load balancer needs a strategy to answer:

"Which backend server should receive this request?"

This strategy is called a **Load Balancing Algorithm**.

There are two broad categories:

Load Balancing Algorithms



14. Static Algorithms

Static algorithms don't continuously consider the current state/load of servers.

Common examples:

1. Round Robin
2. Weighted Round Robin
3. IP Hash

15. Round Robin

Requests are distributed sequentially.

Suppose:

Servers:

S1
S2
S3

Requests:

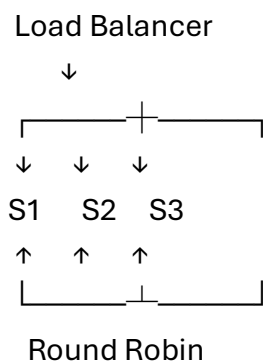
Request 1 → S1
Request 2 → S2
Request 3 → S3

Request 4 → S1

Request 5 → S2

Request 6 → S3

Flow:



Use case

Works well when servers have:

- Similar capacity
- Similar workloads
- Similar request-processing costs

16. Weighted Round Robin

Not all servers have the same capacity.

Suppose:

S1 → Weight 5

S2 → Weight 3

S3 → Weight 2

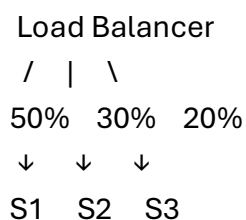
The load balancer sends approximately:

S1 → 50%

S2 → 30%

S3 → 20%

Conceptually:



Use case

Useful when servers have different capacities.

For example:

S1 → 16 CPU

S2 → 8 CPU

S3 → 4 CPU

S1 can handle more traffic.

17. IP Hash

The load balancer uses the client's IP address to determine the server.

Conceptually:

Client IP

↓

Hash Function

↓

Server Selection

Example:

IP A → Hash → S1

IP B → Hash → S3

IP C → Hash → S2

The same client IP will generally map to the same server as long as the server set/hash mapping remains stable.

Why is this useful?

It can provide **session stickiness**.

Example:

User A

↓

S1

Later:

User A



S1

This can be useful when application state is stored locally on the server.

Important

In modern distributed systems, it's usually better to avoid depending heavily on local server session state and instead use shared storage such as a database/cache when appropriate.

PART 6 — DYNAMIC LOAD BALANCING

Dynamic algorithms consider the **current state of servers**.

Load Balancer



Check server state



Choose less-loaded server

Common algorithms:

- Least Connections
- Weighted Least Connections
- Least Response Time

18. Least Connections

The request goes to the server having the fewest active connections.

Suppose:

S1 → 50 connections

S2 → 20 connections

S3 → 35 connections

New request:

New Request



Load Balancer

↓

S2 selected

because S2 currently has the fewest connections.

19. Weighted Least Connections

This combines:

- Number of connections
- Server capacity/weight

Example:

S1 → powerful → weight 5

S2 → medium → weight 3

S3 → weak → weight 1

The algorithm considers both:

Current connections

+

Server capacity

So a powerful server can handle more connections.

20. Least Response Time

The load balancer sends traffic toward the server that is responding fastest, often considering response time and sometimes active connections depending on the implementation.

Example:

S1 → 200 ms

S2 → 80 ms

S3 → 150 ms

New request:

Request

↓

Load Balancer

↓
S2

because S2 currently has the lowest response time.

Use case

Useful when server response times differ significantly.

21. Static vs Dynamic Algorithms

Static	Dynamic
Doesn't continuously use current server load	Uses current server state
Simple	More complex
Lower overhead	Higher overhead
Round Robin	Least Connections
Weighted Round Robin	Weighted Least Connections
IP Hash	Least Response Time

PART 7 — AVAILABILITY ZONES & REGIONS

Your handwritten notes mention:

Availability Zone and Availability Region

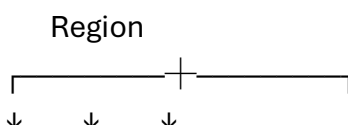
This is important for understanding how large systems remain available.

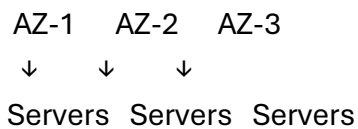
22. Availability Zone

An **Availability Zone (AZ)** is an isolated infrastructure location within a cloud region.

A production application can run across multiple AZs.

Conceptually:





If AZ-1 has a major failure:

AZ-1 ❌

AZ-2 ✅

AZ-3 ✅

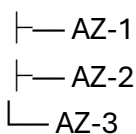
Traffic can continue through healthy zones.

23. Region

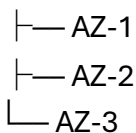
A **region** is a geographic area containing cloud infrastructure.

For example:

Region A



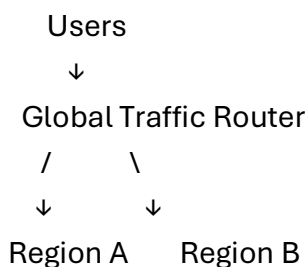
Region B



Running a system across multiple regions can provide stronger disaster resilience, although it adds significant complexity.

24. Traffic Routing During Failure

Suppose:





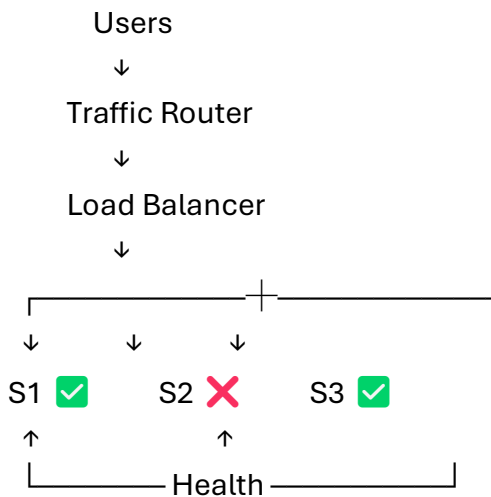
If Region A is unavailable, traffic can be directed toward Region B.

This is the basic idea behind **global traffic management / failover routing**.

Don't go too deep into this yet—you'll encounter availability and fault tolerance again later.

PART 8 — HEALTH CHECK + FAILOVER

Combine the concepts:



The load balancer periodically checks server health.

If:

S2 ✗

then:

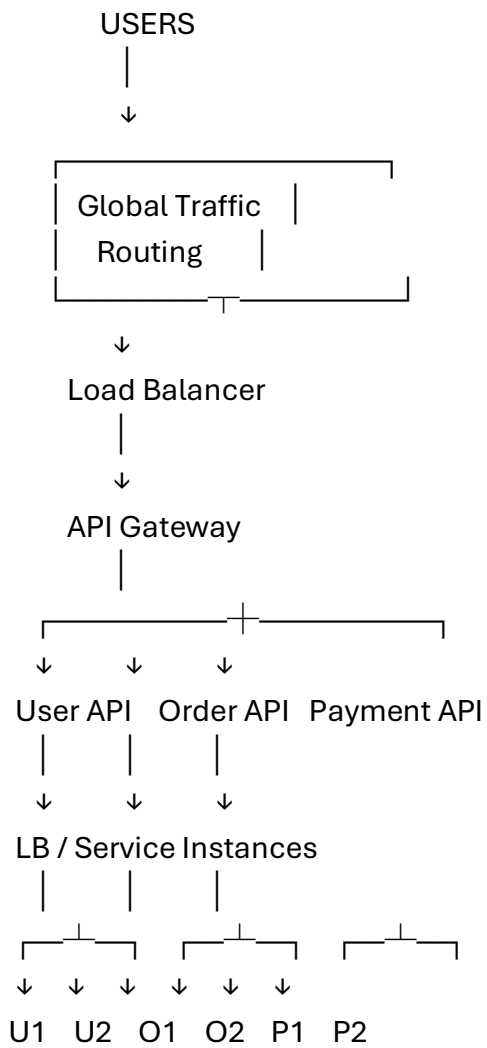
Traffic → S1 + S3

instead of:

Traffic → S1 + S2 + S3

PART 9 — COMPLETE ARCHITECTURE

This is the most useful diagram to understand everything you've learned in Videos 3 & 4.



The responsibilities are roughly:

Global Traffic Routing



Which region?

Load Balancer



Which instance?

API Gateway



Which API/service?

Service



Business logic

This separation is **extremely useful to remember**.

QUICK REVISION TABLE

Component	Main Question It Answers
Global Traffic Routing	Which region should receive traffic?
Load Balancer	Which instance/server should handle it?
API Gateway	Which API/service should handle it?
Service Discovery	Where can I find that service?
Health Check	Is this instance healthy?

ONE EXAMPLE TO REMEMBER EVERYTHING

Imagine you have an e-commerce application.

User requests:

GET /orders/123

Step 1 — User sends request

Mobile App



GET /orders/123

Step 2 — Traffic routing

If you have multiple regions:

Global Router



India Region

Step 3 — Load Balancer

Load Balancer



API Gateway Instance 2

Step 4 — API Gateway

It sees:

/orders/123

and routes it to:

Order Service

Step 5 — Service Load Balancing

Suppose Order Service has:

O1

O2

O3

The load balancer selects one:

Order Service LB

↓

O2

Complete flow

User

↓

Global Traffic Router

↓

Load Balancer

↓

API Gateway

↓

Order Service Load Balancer

↓

Order Service Instance

↓

Database

WHICH LOAD-BALANCING ALGORITHM SHOULD I USE?

Don't memorize a single "best" algorithm.

Think about the situation.

Similar servers + similar requests

→ **Round Robin**

Different server capacities

→ **Weighted Round Robin**

Need client affinity based on IP

→ **IP Hash**

Different number of active connections

→ **Least Connections**

Different server capacities + different connections

→ **Weighted Least Connections**

Servers have significantly different response times

→ **Least Response Time**



MUST KNOW FOR HLD INTERVIEWS

After Videos 3 & 4, you should be able to explain:

API Gateway

- What is an API Gateway?
- Why do we need it?
- Request routing
- Authentication vs Authorization
- Request aggregation
- Rate limiting — basic idea
- SSL/TLS termination — basic idea
- Logging/monitoring

Load Balancer

- What is a Load Balancer?
- Why do we need it?

- Horizontal scaling
- High availability
- Health checks
- Failover
- L4 vs L7

Algorithms

- Round Robin
- Weighted Round Robin
- IP Hash
- Least Connections
- Weighted Least Connections
- Least Response Time

Architecture

You should understand:

Client

↓

Traffic Routing

↓

Load Balancer

↓

API Gateway

↓

Service

↓

Service Instances

And understand that **the exact placement can vary depending on the architecture.**

Important Concepts You Have NOT Studied Yet

Don't go deep into these now because they're coming later:

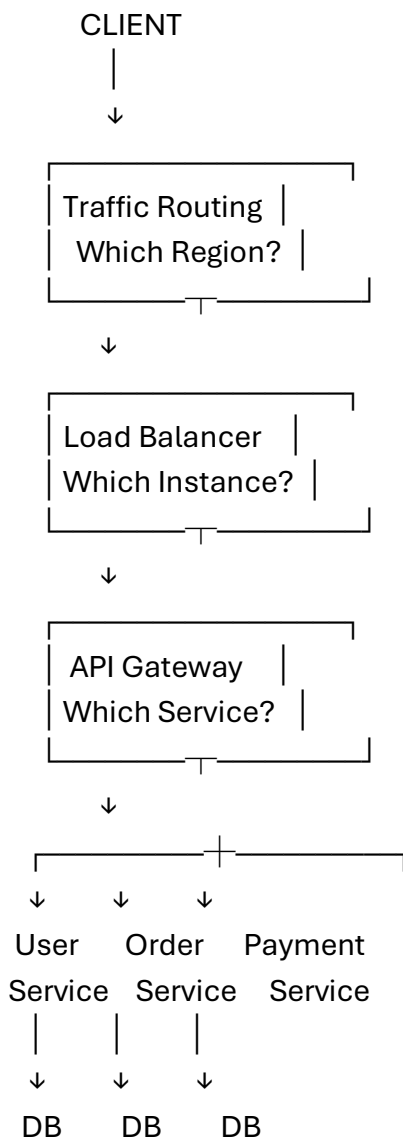
- Caching → **Video #10**
- Distributed systems → **Video #11**
- CDN → **Video #12**
- Rate limiting → **Videos #14–17**

- SSL/TLS → **Video #16**
- Consistent hashing → **Video #21**
- Fault tolerance/SPOF → **Video #19**

You only need their **basic connection** right now.

FINAL MEMORY MAP

For revision, remember this:



The simplest way to remember the whole topic:

Load Balancer distributes traffic.

API Gateway manages and routes APIs.

L4 works mainly with network/transport information.

L7 understands application-level requests such as HTTP.

Static algorithms don't depend heavily on current server state.

Dynamic algorithms consider current server conditions.

Health checks prevent traffic from being sent to unhealthy instances.